

BlockPro App — API Calls

This document describes every API call the BlockPro Flutter app makes against the Bubble.io backend (the v2 API, introduced in commit `f29beef`).

Overview

Item	Detail
Base URL	From the <code>BUBBLE_API_BASE_URL</code> env var (<code>.env</code>). Points at the Bubble workflow API ending in <code>.../api/1.1/wf/</code>
HTTP client	Dart <code>http</code> package
Auth	Bearer token (JWT) in the <code>Authorization</code> header
Config	api_config.dart — loads base URL from <code>.env</code>
HTTP helpers	api_repository.dart — <code>authenticatedGet</code> , <code>authenticatedGetRaw</code> , <code>authenticatedPost</code>

All authenticated requests send:

```
Authorization: Bearer <token>
Content-Type: application/json
```

Every request reports success/failure to the `ConnectivityService` for online/offline tracking. Non-200 responses and network errors throw.

Endpoints

1. `app_login` — Authenticate

Path	<code>POST app_login</code>
Auth	None
Source	auth_repository.dart

Request:

```
{ "email": "<email>", "password": "<password>" }
```

Response:

```
{
  "status": "success",
  "response": {
    "token": "<bearer-token>",
    "user_id": "<user-id>",
  }
}
```

```
    "expires": 86400
  }
}
```

The token is stored in memory and persisted to `SharedPreferences`. Expiry is computed as `now + expires` seconds; on restart, an expired token logs the user out automatically.

2. `app_fetchbuildings` — List Buildings

Path	GET <code>app_fetchbuildings</code>
Auth	Bearer
Query	None
Source	sync_repository.dart

Called in **Phase 1** of the initial sync (and on retry). Returns a JSON array of buildings, upserted into the local `buildings` table.

```
[ { "id": "1756484476534x490740256824968800", "name": "Building 1" } ]
```

Parsed by `parseBuildingsResponse` in [api_parsers.dart](#), which defensively handles a direct array, a `{ response: [...]}` envelope, or a double-encoded JSON string.

3. `app_fetch_all_assets` — List Assets for a Building

Path	GET <code>app_fetch_all_assets</code>
Auth	Bearer
Query	<code>block_id</code> = building ID
Source	sync_repository.dart

Called in **Phase 2** of the initial sync — one call per building. Returns a JSON array of assets, upserted into the local `assets` table. The response's `checklistlastmodified` per asset drives the incremental checklist sync in Phase 3.

```
[
  {
    "assetId": "1771864899143x375085884294525250",
    "buildingId": "1756831363197x720244312004630400",
    "taskname": "Fire door inspection",
    "assetnickname": "896",
    "assetregisteritems": "",
    "tooltiptext": "",
    "tooltipurls": "",
    "lastcompleted": "2026-04-14T11:27:56.319Z",
    "duedate": "2026-04-14T11:27:56.319Z",
    "frequency": "7 Day(s)",
```

```

    "colour": "Red",
    "location": "",
    "floor": "",
    "yellowdate": "2026-04-14T11:27:56.319Z",
    "assetlastmodified": "2026-04-13T10:09:31.738Z",
    "checklistlastmodified": "2026-04-14T11:02:50.833Z"
  }
]

```

Parsed by `parseAssetsResponse` in [api_parsers.dart](#); fields map to the `Asset` model in [asset.dart](#). `colour` is an enum (`Red` / `Yellow` / `Green`).

4. `app_fetch_checklist_single` — Checklist for an Asset

Path	GET <code>app_fetch_checklist_single</code>
Auth	Bearer
Query	<code>asset_id</code> = asset ID
Source	sync_repository.dart

Called in **Phase 3** of the initial sync — one call per asset, run in parallel with a max concurrency of 5. **Incremental:** a checklist is only fetched when its `checklistlastmodified` differs from the stored value. Returns a JSON array with a single checklist object holding a chapters → questions hierarchy.

```

[
  {
    "parentassetid": "1771864899143x375085884294525250",
    "chapters": [
      {
        "chaptername": "Fire door",
        "chapterorder": 1,
        "questions": [
          {
            "questionid": "1771871963645x392498056957566000",
            "questiontext": "Does this door self-close fully into its frame?",
            "questiondesc": "Additional guidance",
            "answertype": "Yes|No",
            "photorequirement": "Always",
            "questionordernumber": 5,
            "existingremedials": []
          }
        ]
      }
    ]
  }
]

```

Parsed by `parseChecklistResponse` in [api_parsers.dart](#). Chapters are stored via `chaptersDao`, and questions (with any `existingremedials` as a JSON blob) via `questionsDao`. Model definitions live in [question.dart](#).

- **answertype** (enum): Yes|No , Yes|No|N/A , Satisfactory|Unsatisfactory , Satisfactory|Unsatisfactory|N/A
 - **photorequirement** (enum): Always , Only when unsatisfactory
 - **Remedial fields:** remedialname , remedialdesc , remedialallocation , remedialduedate , remedialpriority (Low / High)
-

5. app_upload-image — Upload Inspection Photo

Path	POST app_upload-image
Auth	Bearer
Source	inspection_provider.dart

Called during inspection submission, once per attached photo, **before** the final submit. Each image is read and base64-encoded.

Request:

```
{
  "base64": "<base64-encoded-image-bytes>",
  "asset_id": "<asset-id>",
  "filename": "photo_2026-03-01.jpg"
}
```

Response:

```
{ "response": { "image_id": "img_abc123", "image_url": "<url>" } }
```

The returned `image_id` is collected for the submission payload. Upload failures are logged but non-fatal — submission continues without that photo.

6. app_completed-inspection — Submit Inspection

Path	POST app_completed-inspection
Auth	Bearer
Source	inspection_provider.dart

Called when the user taps Submit, after all photos have uploaded.

Request:

```
{
  "asset_id": "<asset-id>",
  "answers": [
    { "question_text": "Does this door self-close?", "answer_text": "Yes" },
    { "question_text": "Is the seal intact?", "answer_text": "No" }
  ],
}
```

```
"photo_ids": ["img_abc123", "img_def456"]
}
```

- `photo_ids` is included only when at least one photo uploaded successfully.
- `answer_text` is the selected label, or an empty string if unanswered.

Client-side validation before submit: every question with answer options must be answered, and a photo must be attached wherever the `photorequirement` rule (given the selected answer) demands one.

Sync Flow

Orchestrated by `SyncRepository.syncAll()` , driven by `InitialSyncNotifier` .

```
Login
|
v
Phase 1: app_fetchbuildings      —> upsert buildings table
|
v
Phase 2: app_fetch_all_assets    —> one call per building
|                               upsert assets table; capture checklistlastmodified
v
Phase 3: app_fetch_checklist_single —> one call per STALE asset (parallel, max 5)
|                               upsert chapters + questions tables
```

The initial sync screen ([initial_sync_screen.dart](#)) shows progress per phase with a retry button on failure. All UI reads from the local SQLite (Drift) database; the API is used only to populate/refresh that cache.

Summary

#	Endpoint	Method	Auth	Sends	Returns
1	<code>app_login</code>	POST	No	email, password	token, user_id, expires
2	<code>app_fetchbuildings</code>	GET	Bearer	—	building array
3	<code>app_fetch_all_assets</code>	GET	Bearer	<code>block_id</code>	asset array
4	<code>app_fetch_checklist_single</code>	GET	Bearer	<code>asset_id</code>	checklist (chapters/questions)
5	<code>app_upload-image</code>	POST	Bearer	base64, <code>asset_id</code> , filename	<code>image_id</code> , <code>image_url</code>
6	<code>app_completed-inspection</code>	POST	Bearer	<code>asset_id</code> , <code>answers[]</code> , <code>photo_ids[]</code>	status

Key Files

File	Purpose
------	---------

api_config.dart	Base URL from .env
api_repository.dart	Authenticated GET/POST helpers
auth_repository.dart	Login, token storage, session
sync_repository.dart	Buildings / assets / checklists sync
api_parsers.dart	Response parsing & normalization
inspection_provider.dart	Photo upload & inspection submit